SLIOR – Sistema de Optimización de Rutas Logísticas

Documentación Técnica del Proyecto CFGS DAM

---

1. DESCRIPCIÓN DEL SISTEMA

1.1. Visión General

SLIOR es una aplicación móvil Android diseñada para optimizar la logística de última milla, permitiendo a repartidores y pequeñas empresas de mensajería planificar rutas eficientes, gestionar entregas y reducir costes operativos.

1.2. Problema que Resuelve

- **Rutas no optimizadas**: Pérdida de tiempo y combustible por planificación manual
- **Conectividad limitada**: Los repartidores trabajan en zonas con mala cobertura
- **Entrada manual de datos**: Errores y lentitud al registrar paquetes
- **Falta de visibilidad**: Empresas sin control en tiempo real de sus entregas

1.3. Solución Propuesta

Sistema multiplataforma (móvil + backend) con enfoque **offline-first** que permite:

- Planificación automática de rutas optimizadas
- Gestión de paquetes mediante escaneo de códigos
- Funcionamiento sin conexión continua
- Sincronización inteligente de datos

1.4. Usuarios Objetivo

- Repartidores autónomos
- Pequeñas empresas de mensajería (2-20 vehículos)
- Negocios de e-commerce con reparto propio
- Empresas de courier local

---

2. ARQUITECTURA DEL SOFTWARE

2.1. Arquitectura General del Sistema

```
┌─────────────────────────────────────────────────────────────┐
│                    APLICACIÓN MÓVIL                         │
│                      (Android)                              │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  Capa de Presentación (UI)                          │   │
│  │  Activities, Fragments, Jetpack Compose             │   │
│  └─────────────────────────────────────────────────────┘   │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  Capa de Lógica (ViewModels)                        │   │
│  │  LiveData, Flow, Casos de Uso                       │   │
│  └─────────────────────────────────────────────────────┘   │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  Capa de Datos (Repository)                         │   │
│  │  Room (SQLite), Retrofit, DataStore                 │   │
│  └─────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘
                          ↕ HTTP/REST
┌─────────────────────────────────────────────────────────────┐
│                    BACKEND (API)                            │
│                  Spring Boot + Java                         │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  Controladores REST (Controllers)                   │   │
│  └─────────────────────────────────────────────────────┘   │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  Lógica de Negocio (Services)                       │   │
│  └─────────────────────────────────────────────────────┘   │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  Acceso a Datos (Repositories + JPA)                │   │
│  └─────────────────────────────────────────────────────┘   │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  PostgreSQL Database                                │   │
│  └─────────────────────────────────────────────────────┘   │
└─────────────────────────────────────────────────────────────┘
```

2.2. Arquitectura de la App Móvil (MVVM + Clean Architecture)

Patrón MVVM (Model-View-ViewModel):

- **View**: Activities/Fragments/Compose
- **ViewModel**: Gestión de estado UI
- **Model**: Entidades y lógica de datos

Capas:

1. **Presentation Layer**: UI components
2. **Domain Layer**: Casos de uso y lógica de negocio
3. **Data Layer**: Repositorios y fuentes de datos

2.3. Arquitectura Backend (Tres Capas)

```
Controller → Service → Repository → Database
    ↓          ↓          ↓
  DTOs     Business    Entities
           Logic
```

---

3. MODELO DE DATOS

3.1. Entidades Principales

Usuario

- ID (UUID)
- Nombre completo
- Email (único)
- Password (hash)
- Rol (REPARTIDOR, ADMINISTRADOR)
- Fecha de registro
- Estado (ACTIVO, INACTIVO)

Ruta

- ID (UUID)
- Usuario ID (FK)
- Nombre de la ruta
- Fecha de creación
- Fecha de inicio planificada
- Estado (PLANIFICADA, EN_CURSO, COMPLETADA, CANCELADA)
- Distancia total (km)
- Tiempo estimado (minutos)
- Número de paradas

Parada (Stop)

- ID (UUID)
- Ruta ID (FK)
- Dirección completa
- Latitud
- Longitud
- Orden en la ruta (1, 2, 3...)
- Tiempo estimado de llegada
- Estado (PENDIENTE, EN_CAMINO, ENTREGADA, FALLIDA)
- Notas
- Fecha de entrega real

Paquete

- ID (UUID)
- Parada ID (FK)
- Código de barras (único)
- Descripción
- Destinatario
- Email destinatario
- Teléfono destinatario
- Peso (kg)
- Estado (REGISTRADO, EN_TRANSITO, ENTREGADO, DEVUELTO)
- Fecha de registro
- Fecha de entrega
- Firma digital (imagen base64)
- Foto de entrega (opcional)

Sincronización

- ID
- Tipo de entidad (RUTA, PARADA, PAQUETE)
- ID de entidad
- Acción (CREATE, UPDATE, DELETE)
- Timestamp local
- Estado de sincronización (PENDIENTE, SINCRONIZADA, ERROR)
- Datos JSON

---

4. DISEÑO DE LA BASE DE DATOS

4.1. Diagrama Relacional (Backend - PostgreSQL)

sql

```sql
-- Tabla: usuarios
CREATE TABLE usuarios (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre_completo VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    rol VARCHAR(20) NOT NULL CHECK (rol IN ('REPARTIDOR', 'ADMINISTRADOR')),
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'ACTIVO'
);

-- Tabla: rutas
CREATE TABLE rutas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    nombre VARCHAR(100) NOT NULL,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```sql
    fecha_inicio_planificada TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'PLANIFICADA',
    distancia_total DECIMAL(10,2),
    tiempo_estimado INTEGER,
    numero_paradas INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla: paradas
CREATE TABLE paradas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ruta_id UUID NOT NULL REFERENCES rutas(id) ON DELETE CASCADE,
    direccion VARCHAR(255) NOT NULL,
    latitud DECIMAL(10,8) NOT NULL,
    longitud DECIMAL(11,8) NOT NULL,
    orden_ruta INTEGER NOT NULL,
    tiempo_estimado_llegada TIMESTAMP,
    estado VARCHAR(20) DEFAULT 'PENDIENTE',
    notas TEXT,
    fecha_entrega_real TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla: paquetes
CREATE TABLE paquetes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    parada_id UUID NOT NULL REFERENCES paradas(id) ON DELETE CASCADE,
    codigo_barras VARCHAR(100) UNIQUE NOT NULL,
    descripcion VARCHAR(255),
    destinatario VARCHAR(100) NOT NULL,
    email_destinatario VARCHAR(100),
    telefono_destinatario VARCHAR(20),
    peso DECIMAL(6,2),
    estado VARCHAR(20) DEFAULT 'REGISTRADO',
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_entrega TIMESTAMP,
    firma_digital TEXT,
    foto_entrega TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Índices para optimización
CREATE INDEX idx_rutas_usuario ON rutas(usuario_id);
CREATE INDEX idx_rutas_estado ON rutas(estado);
CREATE INDEX idx_paradas_ruta ON paradas(ruta_id);
CREATE INDEX idx_paradas_orden ON paradas(ruta_id, orden_ruta);
CREATE INDEX idx_paquetes_parada ON paquetes(parada_id);
```

```
CREATE INDEX idx_paquetes_codigo ON paquetes(codigo_barras);
```

4.2. Base de Datos Local (Móvil - Room/SQLite)

Estructura similar con ajustes para offline:

kotlin
```
@Entity(tableName = "rutas_local")
data class RutaLocal(
    @PrimaryKey val id: String,
    val usuarioId: String,
    val nombre: String,
    val estado: String,
    val distanciaTotal: Double?,
    val tiempoEstimado: Int?,
    val sincronizado: Boolean = false,
    val timestamp: Long = System.currentTimeMillis()
)

// Similar para Paradas y Paquetes con flag de sincronización
```

---

5. DISEÑO DE API REST

5.1. Endpoints Principales

Base URL: `https://api.slior.com/v1`

Autenticación
```
POST   /auth/register        # Registro de usuario
POST   /auth/login           # Login (devuelve JWT token)
POST   /auth/refresh         # Refresh token
POST   /auth/logout          # Logout
```

Rutas
```
GET    /rutas                # Listar rutas del usuario
GET    /rutas/{id}           # Obtener ruta específica
POST   /rutas                # Crear nueva ruta
PUT    /rutas/{id}           # Actualizar ruta
DELETE /rutas/{id}           # Eliminar ruta
POST   /rutas/{id}/optimize  # Optimizar orden de paradas
GET    /rutas/{id}/paradas   # Obtener paradas de una ruta
```

Paradas
```
GET    /paradas/{id}          # Obtener parada
POST   /paradas               # Crear parada
PUT    /paradas/{id}          # Actualizar parada
DELETE /paradas/{id}          # Eliminar parada
PUT    /paradas/{id}/estado   # Actualizar estado
```

Paquetes
```
GET    /paquetes              # Listar paquetes
GET    /paquetes/{id}         # Obtener paquete
GET    /paquetes/barcode/{code} # Buscar por código
POST   /paquetes              # Registrar paquete
PUT    /paquetes/{id}         # Actualizar paquete
PUT    /paquetes/{id}/entregar # Marcar como entregado
```

Sincronización
```
POST   /sync/pull             # Descargar cambios del servidor
POST   /sync/push             # Enviar cambios locales
GET    /sync/status           # Estado de sincronización
```

5.2. Ejemplos de Peticiones/Respuestas

POST /rutas

```json
{
  "nombre": "Ruta Centro - 10/03/2026",
  "fechaInicioPlanificada": "2026-03-10T08:00:00Z",
  "paradas": [
    {
      "direccion": "Calle Mayor 15, Herencia",
      "latitud": 39.3655,
      "longitud": -3.3644,
      "notas": "Llamar al timbre 2B"
    },
    {
      "direccion": "Avenida Constitución 42, Herencia",
      "latitud": 39.3622,
      "longitud": -3.3589
    }
  ]
}
```

Respuesta 201 Created

json

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "nombre": "Ruta Centro - 10/03/2026",
  "estado": "PLANIFICADA",
  "numeroParadas": 2,
  "distanciaTotal": 2.8,
  "tiempoEstimado": 15,
  "paradas": [
    {
      "id": "123e4567-e89b-12d3-a456-426614174000",
      "ordenRuta": 1,
      "direccion": "Calle Mayor 15, Herencia",
      "estado": "PENDIENTE"
    },
    {
      "id": "223e4567-e89b-12d3-a456-426614174001",
      "ordenRuta": 2,
      "direccion": "Avenida Constitución 42, Herencia",
      "estado": "PENDIENTE"
    }
  ]
}
```


POST /rutas/{id}/optimize

json

```
{
  "criterio": "DISTANCIA_MINIMA",
  "puntoInicio": {
    "latitud": 39.3700,
    "longitud": -3.3700
  }
}
```


Respuesta optimizada

json

```
{
  "rutaOptimizada": [
    {"paradaId": "223e4567...", "ordenNuevo": 1},
    {"paradaId": "123e4567...", "ordenNuevo": 2}
  ],
  "distanciaTotal": 2.5,
  "tiempoEstimado": 12,
  "ahorro": {
```

```
    "distancia": 0.3,
    "tiempo": 3
  }
}
```

---

6. FUNCIONALIDADES DEL MVP (Versión 1.0)

6.1. Funcionalidades Esenciales

Módulo de Autenticación
- ✅ Registro de usuario
- ✅ Login con email/password
- ✅ Almacenamiento seguro de token JWT

Módulo de Gestión de Rutas
- ✅ Crear ruta nueva
- ✅ Añadir paradas manualmente (dirección + coordenadas)
- ✅ Ver lista de rutas
- ✅ Ver detalle de ruta con mapa
- ✅ Eliminar ruta

Módulo de Optimización Básica
- ✅ Algoritmo simple de optimización (Nearest Neighbor)
- ✅ Ordenar paradas por distancia desde punto actual
- ✅ Calcular distancia y tiempo estimado total

Módulo de Navegación
- ✅ Visualizar ruta en mapa (OpenStreetMap / Mapbox)
- ✅ Mostrar ubicación actual del repartidor
- ✅ Seguir navegación parada por parada
- ✅ Marcar parada como "En camino" / "Completada"

Módulo de Gestión de Paquetes
- ✅ Registrar paquete manualmente
- ✅ Escanear código de barras (usando librería ZXing)
- ✅ Asociar paquete a parada
- ✅ Marcar paquete como entregado
- ✅ Ver listado de paquetes por ruta

Módulo Offline-First
- ✅ Almacenamiento local con Room (SQLite)
- ✅ Funcionamiento completo sin conexión
- ✅ Sincronización manual (botón "Sincronizar")
- ✅ Indicador visual de estado de sincronización

Módulo de Notificaciones
- ✅ Confirmación de entrega por email (usando JavaMail API)

6.2. Flujos de Usuario Principales

Flujo 1: Crear y Ejecutar Ruta
1. Usuario inicia sesión
2. Crea nueva ruta
3. Añade 5 direcciones de entrega
4. Sistema optimiza automáticamente el orden
5. Usuario inicia la ruta
6. Navega parada por parada
7. Marca cada entrega como completada
8. Sincroniza datos al final del día

Flujo 2: Gestionar Paquete
1. Usuario escanea código de barras
2. Sistema busca paquete o permite crear nuevo
3. Asocia paquete a parada actual
4. Al entregar, toma foto (opcional)
5. Marca como entregado
6. Sistema envía email de confirmación

---

7. FUNCIONALIDADES FUTURAS (Post-MVP)

7.1. Versión 1.1 (Si hay tiempo en el proyecto)
- 🔄 Sincronización automática en segundo plano
- 📊 Dashboard con estadísticas (km recorridos, entregas del mes)
- 🔔 Notificaciones push (OneSignal)
- 📸 Firma digital del destinatario en pantalla

7.2. Versión 2.0 (Evolución futura)
- 🧠 Algoritmo avanzado de optimización (VROOM, Genetic Algorithms)
- ⛽ Optimización por consumo de combustible
- 👥 Gestión multi-usuario (admin asigna rutas a repartidores)
- 📍 Tracking en tiempo real para clientes
- 🌐 Aplicación web de administración
- 💬 Chat entre repartidor y cliente
- 🚗 Integración con Google Maps Directions API
- 📈 Análisis predictivo de tiempos de entrega
- 🔐 Autenticación biométrica (huella, Face ID)

---

8. ORGANIZACIÓN DEL REPOSITORIO GIT

8.1. Estructura de Directorios

```
slior-project/
│
├── backend/                    # Backend Spring Boot
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/slior/
│   │   │   │   ├── config/       # Configuración (Security, CORS, etc.)
│   │   │   │   ├── controller/   # REST Controllers
│   │   │   │   ├── service/      # Lógica de negocio
│   │   │   │   ├── repository/   # JPA Repositories
│   │   │   │   ├── model/        # Entidades JPA
│   │   │   │   ├── dto/          # Data Transfer Objects
│   │   │   │   ├── security/     # JWT, Auth
│   │   │   │   ├── exception/    # Manejo de excepciones
│   │   │   │   └── util/         # Utilidades
│   │   │   └── resources/
│   │   │       ├── application.properties
│   │   │       └── schema.sql
│   │   └── test/                 # Tests unitarios e integración
│   ├── pom.xml                   # Dependencias Maven
│   └── README.md
│
├── mobile-app/                   # Aplicación Android
│   ├── app/
│   │   ├── src/
│   │   │   ├── main/
│   │   │   │   ├── java/com/slior/
│   │   │   │   │   ├── ui/       # Activities, Fragments, Compose
│   │   │   │   │   │   ├── auth/
│   │   │   │   │   │   ├── routes/
│   │   │   │   │   │   ├── packages/
│   │   │   │   │   │   └── map/
│   │   │   │   │   ├── viewmodel/   # ViewModels
│   │   │   │   │   ├── data/     # Repository, DataSource
│   │   │   │   │   │   ├── local/   # Room Database
│   │   │   │   │   │   ├── remote/  # Retrofit API
│   │   │   │   │   │   └── repository/
│   │   │   │   │   ├── domain/   # Casos de uso
│   │   │   │   │   ├── model/    # Modelos de datos
│   │   │   │   │   ├── di/       # Dependency Injection (Hilt)
│   │   │   │   │   └── util/     # Utilidades, extensiones
│   │   │   │   ├── res/          # Recursos (layouts, drawables, etc.)
│   │   │   │   └── AndroidManifest.xml
│   │   │   └── test/
│   │   └── build.gradle.kts
│   ├── gradle/
```

```
│   ├── build.gradle.kts
│   └── README.md
│
├── docs/                       # Documentación del proyecto
│   ├── arquitectura.md
│   ├── api-specification.md
│   ├── database-design.md
│   ├── diagramas/
│   │   ├── arquitectura-sistema.png
│   │   ├── modelo-datos.png
│   │   ├── flujo-usuario.png
│   │   └── diagrama-clases.png
│   └── manual-usuario.md
│
├── scripts/                    # Scripts útiles
│   ├── setup-db.sql
│   └── seed-data.sql
│
├── .gitignore
├── README.md                   # Documentación principal
└── LICENSE
```

8.2. Archivo .gitignore

gitignore

```
# Backend (Java/Spring Boot)
backend/target/
backend/.mvn/
backend/mvnw
backend/mvnw.cmd
backend/*.log
backend/.idea/
backend/*.iml

# Mobile App (Android)
mobile-app/.gradle/
mobile-app/build/
mobile-app/local.properties
mobile-app/.idea/
mobile-app/*.iml
mobile-app/app/build/
mobile-app/.DS_Store

# Archivos sensibles
*.env
application-prod.properties
```

---

9. FLUJO DE TRABAJO CON GIT

9.1. Estrategia de Branching (Git Flow Simplificado)

Ramas principales:

- `main`: Código en producción (releases)
- `develop`: Rama de desarrollo activa

Ramas de soporte:

- `feature/nombre-funcionalidad`: Para nuevas features
- `bugfix/nombre-bug`: Para correcciones
- `hotfix/nombre-urgente`: Para parches urgentes en producción

9.2. Flujo de Trabajo

1. Iniciar nueva funcionalidad

bash
```bash
git checkout develop
git pull origin develop
git checkout -b feature/optimizacion-rutas
```

2. Desarrollar y commitear

bash
```bash
# Hacer cambios...
git add .
git commit -m "feat: implementar algoritmo de optimización básico"
```

3. Sincronizar con develop

bash
```bash
git checkout develop
git pull origin develop
git checkout feature/optimizacion-rutas
git rebase develop
```

4. Crear Pull Request

bash
```bash
git push origin feature/optimizacion-rutas
# Crear PR en GitHub/GitLab hacia develop
```

5. Merge a develop (tras revisión)

bash

```
git checkout develop
git merge feature/optimizacion-rutas
git push origin develop
```

6. Release

bash
```
git checkout main
git merge develop
git tag -a v1.0.0 -m "Release MVP 1.0"
git push origin main --tags
```

9.3. Convención de Commits (Conventional Commits)
```
<tipo>(<alcance>): <descripción>

[cuerpo opcional]

[nota al pie opcional]
```

Tipos:

- `feat`: Nueva funcionalidad
- `fix`: Corrección de bug
- `docs`: Cambios en documentación
- `style`: Formato, sin cambios de código
- `refactor`: Refactorización
- `test`: Añadir tests
- `chore`: Tareas de mantenimiento

Ejemplos:

bash
```
feat(auth): añadir endpoint de registro de usuario
fix(maps): corregir crash al cargar mapa sin conexión
docs(api): actualizar documentación de endpoints
refactor(database): optimizar consulta de rutas
```

---

10. PLAN DE DESARROLLO POR FASES

FASE 1: Configuración y Fundamentos (15 horas)

Semana 1-2

Backend (8 horas)
- ✅ Configurar proyecto Spring Boot
- ✅ Configurar PostgreSQL y crear esquema inicial
- ✅ Implementar modelos JPA básicos (Usuario, Ruta, Parada)
- ✅ Configurar Spring Security + JWT
- ✅ Implementar endpoints de autenticación

Mobile App (7 horas)
- ✅ Configurar proyecto Android con Kotlin
- ✅ Configurar Room Database (esquema local)
- ✅ Configurar Retrofit para API
- ✅ Implementar inyección de dependencias (Hilt)
- ✅ Crear pantalla de login/registro

FASE 2: Gestión de Rutas (25 horas)

Semana 3-4

Backend (12 horas)
- ✅ Implementar CRUD de rutas
- ✅ Implementar CRUD de paradas
- ✅ Crear servicio de optimización básico (Nearest Neighbor)
- ✅ Endpoint de optimización de rutas
- ✅ Tests unitarios de servicios

Mobile App (13 horas)
- ✅ Implementar Repository Pattern
- ✅ ViewModels para rutas y paradas
- ✅ Pantalla de lista de rutas
- ✅ Pantalla de creación de ruta
- ✅ Pantalla de detalle de ruta
- ✅ Integración con backend (Retrofit)
- ✅ Almacenamiento local (Room)

FASE 3: Mapas y Navegación (20 horas)

Semana 5-6

Mobile App (20 horas)
- ✅ Integrar librería de mapas (Mapbox/OSMDroid)
- ✅ Mostrar ruta en mapa con marcadores
- ✅ Implementar geolocalización (LocationManager)
- ✅ Navegación paso a paso
- ✅ Actualizar estado de paradas (En camino/Completada)

- ✅ Optimización de rendimiento del mapa
- ✅ Modo offline para mapas (cache de tiles)

FASE 4: Gestión de Paquetes (15 horas)

Semana 7

Backend (6 horas)
- ✅ Implementar modelo y endpoints de Paquetes
- ✅ Asociación paquete-parada
- ✅ Envío de emails (JavaMail)

Mobile App (9 horas)
- ✅ Integrar ZXing para escaneo de códigos
- ✅ Pantalla de escaneo de paquetes
- ✅ Pantalla de listado de paquetes
- ✅ Formulario de registro de paquete
- ✅ Marcar paquete como entregado
- ✅ Captura de foto de entrega (opcional MVP)

FASE 5: Sincronización Offline (12 horas)

Semana 8

Mobile App (10 horas)
- ✅ Implementar lógica de sincronización delta
- ✅ Detección de conectividad (NetworkCallback)
- ✅ Queue de operaciones pendientes
- ✅ Resolución de conflictos (timestamp)
- ✅ UI de indicadores de sincronización
- ✅ Sincronización manual (pull-to-refresh)

Backend (2 horas)
- ✅ Endpoint de sincronización por lotes
- ✅ Gestión de timestamps

FASE 6: Testing y Refinamiento (13 horas)

Semana 9-10

Backend (5 horas)
- ✅ Tests de integración de API

- ✅ Tests de servicios de optimización
- ✅ Corrección de bugs

Mobile App (8 horas)
- ✅ Tests unitarios de ViewModels
- ✅ Tests de Repository
- ✅ Tests UI con Espresso (básicos)
- ✅ Optimización de rendimiento
- ✅ Pulido de UI/UX
- ✅ Corrección de bugs
- ✅ Pruebas en dispositivo real

TOTAL: ~100 horas

Distribución:
- Backend: ~33 horas
- Mobile App: ~57 horas
- Testing: ~13 horas (integrado en fases)

---

11. DIAGRAMA DE MÓDULOS DEL SISTEMA

11.1. Módulos del Backend
```
┌─────────────────────────────────────────────────────────────┐
│                    SPRING BOOT BACKEND                      │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  MÓDULO DE SEGURIDAD                                │   │
│  │  - JWT Authentication                               │   │
│  │  - Spring Security Config                           │   │
│  │  - UserDetailsService                               │   │
│  └─────────────────────────────────────────────────────┘   │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  MÓDULO DE GESTIÓN DE USUARIOS                      │   │
│  │  - UserController                                   │   │
│  │  - UserService                                      │   │
│  │  - UserRepository                                   │   │
│  └─────────────────────────────────────────────────────┘   │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐   │
│  │  MÓDULO DE GESTIÓN DE RUTAS                         │   │
│  │  - RouteController                                  │   │
```

```
│  - RouteService                    │  │
│  - RouteRepository                 │  │
│  - RouteOptimizationService        │  │
└──────────────────────────────────────┘  │
                    │                     │
┌──────────────────────────────────────┐  │
│  MÓDULO DE GESTIÓN DE PARADAS        │  │
│  - StopController                  │  │
│  - StopService                     │  │
│  - StopRepository                  │  │
└──────────────────────────────────────┘  │
                    │                     │
┌──────────────────────────────────────┐  │
│  MÓDULO DE GESTIÓN DE PAQUETES       │  │
│  - PackageController               │  │
│  - PackageService                  │  │
│  - PackageRepository               │  │
│  - EmailService                    │  │
└──────────────────────────────────────┘  │
                    │                     │
┌──────────────────────────────────────┐  │
│  MÓDULO DE SINCRONIZACIÓN            │  │
│  - SyncController                  │  │
│  - SyncService                     │  │
└──────────────────────────────────────┘  │
                    │                     │
└──────────────────────────────────────────┘
```

11.2. Módulos de la App Móvil

```
┌──────────────────────────────────────────┐
│      APLICACIÓN ANDROID (KOTLIN)         │
├──────────────────────────────────────────┤
│                    │                     │
┌──────────────────────────────────────┐  │
│      CAPA DE PRESENTACIÓN (UI)       │  │
│                    │  │              │  │
┌──────────────────────────────────┐  │  │
│  Módulo de Autenticación         │  │  │
│  - LoginActivity                 │  │  │
│  - RegisterActivity              │  │  │
│  - AuthViewModel                 │  │  │
└──────────────────────────────────┘  │  │
                    │  │              │  │
┌──────────────────────────────────┐  │  │
│  Módulo de Rutas                 │  │  │
│  - RoutesListActivity            │  │  │
```

```
│ │ │ - RouteDetailActivity          │ │   │
│ │ │ - CreateRouteActivity          │ │   │
│ │ │ - RouteViewModel               │ │   │
│ │ └──────────────────────────────────┘ │ │
│ │                │ │
│ │ ┌──────────────────────────────────┐ │ │
│ │ │ Módulo de Mapa y Navegación    │ │   │
│ │ │ - MapActivity                  │ │   │
│ │ │ - NavigationFragment           │ │   │
│ │ │ - MapViewModel                 │ │   │
│ │ └──────────────────────────────────┘ │ │
│ │                │ │
│ │ ┌──────────────────────────────────┐ │ │
│ │ │ Módulo de Paquetes             │ │   │
│ │ │ - PackageListActivity          │ │   │
│ │ │ - ScanBarcodeActivity          │ │   │
│ │ │ - PackageDetailActivity        │ │   │
│ │ │ - PackageViewModel             │ │   │
│ │ └──────────────────────────────────┘ │ │
│ └────────────────────────────────────────┘ │
│                │
│ ┌────────────────────────────────────────┐ │
│ │     CAPA DE DOMINIO            │ │   │
│ │                │ │
│ │ - GetRoutesUseCase             │ │   │
│ │ - OptimizeRouteUseCase         │ │   │
│ │ - SyncDataUseCase              │ │   │
│ │ - ScanPackageUseCase           │ │   │
│ └────────────────────────────────────────┘ │
│                │
│ ┌────────────────────────────────────────┐ │
│ │     CAPA DE DATOS              │ │   │
│ │                │ │
│ │ ┌──────────────────────────────────┐ │ │
│ │ │ Repositorios                   │ │   │
│ │ │ - RouteRepository              │ │   │
│ │ │ - PackageRepository            │ │   │
│ │ │ - UserRepository               │ │   │
│ │ │ - SyncRepository               │ │   │
│ │ └──────────────────────────────────┘ │ │
│ │                │ │
│ │ ┌──────────────────────────────────┐ │ │
│ │ │ Fuente de Datos Local (Room)   │ │   │
│ │ │ - RouteDao                     │ │   │
│ │ │ - PackageDao                   │ │   │
│ │ │ - UserDao                      │ │   │
│ │ │ - AppDatabase                  │ │   │
│ │ └──────────────────────────────────┘ │ │
```

---

12. TECNOLOGÍAS JUSTIFICADAS

12.1. Backend

Tecnología	Versión	Justificación
Java	17 LTS	Lenguaje requerido por proyecto DAM. LTS garantiza soporte a largo plazo.
Spring Boot	3.2.x	Framework estándar de industria. Simplifica configuración y desarrollo de APIs REST.
Spring Security	-	Autenticación y autorización robusta. Integración JWT out-of-the-box.
PostgreSQL	15+	Base de datos relacional robusta, gratuita, con excelente rendimiento. Soporta tipos geoespaciales (PostGIS) para futuras mejoras.
Maven	3.9+	Gestor de dependencias estándar en ecosistema Java.
Hibernate (JPA)	-	ORM incluido en Spring Boot. Reduce código boilerplate de SQL.
JWT (jjwt)	0.12.x	Autenticación stateless, ideal para apps móviles.
JavaMail API	-	Envío de emails de confirmación de entrega.
Lombok	-	Reduce código repetitivo (getters, setters, constructores).

12.2. Aplicación Móvil

Tecnología	Versión	Justificación
Kotlin	1.9+	Lenguaje oficial de Android. Más conciso y seguro que Java.
Android SDK	API 24+ (Android 7.0)	Cubre ~95% de dispositivos activos.
Jetpack Compose	1.6+	UI declarativa moderna (opcional, puede usarse XML también).
Room Database	2.6+	Abstracción sobre SQLite. Parte de Android Jetpack.
Retrofit	2.9+	Cliente HTTP estándar para Android. Fácil integración con API REST.
Hilt	2.48+	Inyección de dependencias recomendada por Google.
Coroutines	1.7+	Manejo de asincronía eficiente y legible.
LiveData / Flow	-	Observables para UI reactiva.
Mapbox SDK	11.x	Mapas personalizables, 25.000 cargass de mapa gratis/mes. Alternativa: OSMDroid (100% gratis).
ZXing (Zebra Crossing)	3.5+	Librería open-source para escaneo de códigos. Probada y estable.
Coil	2.5+	Carga eficiente de imágenes (fotos de entregas).
WorkManager	2.9+	Ejecución de tareas en segundo plano (sincronización).

12.3. Control de Versiones y DevOps

Herramienta	Justificación
Git	Control de versiones estándar.
GitHub / GitLab	Hosting de repositorios. GitHub más popular, GitLab ofrece CI/CD gratis.
GitHub Actions / GitLab CI	Automatización de tests y builds (opcional para MVP).

---

13. POSIBLES MEJORAS FUTURAS

13.1. Optimización Avanzada

- **Algoritmo VROOM**: Motor C++ de código abierto para VRP real
- **Google OR-Tools**: Librería de optimización de Google
- **Algoritmos genéticos**: Para optimización multiobjetivo (tiempo + combustible + tráfico)
- **Machine Learning**: Predicción de tiempos de entrega basada en históricos

13.2. Funcionalidades Empresariales

- **Multi-tenant**: Soporte para múltiples empresas en la misma plataforma
- **Panel de administración web**: Dashboard con React/Angular
- **Asignación automática de rutas**: IA que asigna rutas a repartidores según capacidad
- **Optimización de flota**: Gestión de múltiples vehículos simultáneos

13.3. Experiencia de Usuario

- **Tracking en tiempo real**: Clientes pueden ver dónde está su paquete
- **Chatbot**: Asistente virtual para repartidores
- **Modo manos libres**: Control por voz durante conducción
- **Realidad Aumentada**: AR para encontrar direcciones exactas

13.4. Integraciones

- **Waze API**: Datos de tráfico en tiempo real
- **What3Words**: Direccionamiento preciso con 3 palabras
- **WhatsApp Business API**: Notificaciones por WhatsApp
- **Zapier / Make**: Integraciones no-code con otros sistemas

13.5. Analytics y BI

- **Dashboard de métricas**: Tiempo promedio de entrega, km por día, consumo
- **Reportes automáticos**: PDF con estadísticas mensuales
- **Gamificación**: Sistema de puntos y rankings para repartidores

13.6. Infraestructura

- **Dockerización**: Contenedores para fácil despliegue
- **Kubernetes**: Orquestación para escalar automáticamente
- **Cloud deployment**: AWS / GCP / Azure
- **CDN**: Para imágenes y tiles de mapa

ANEXOS

Anexo A: Librerías Clave Android

kotlin

```kotlin
// build.gradle.kts (app)
dependencies {
    // Core
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")

    // UI
    implementation("com.google.android.material:material:1.11.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")

    // Jetpack Compose (Opcional)
    implementation("androidx.compose.ui:ui:1.6.0")
    implementation("androidx.compose.material3:material3:1.1.2")

    // Architecture Components
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.7.0")
    implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")

    // Room
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    kapt("androidx.room:room-compiler:2.6.1")

    // Retrofit
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

    // Hilt
    implementation("com.google.dagger:hilt-android:2.48")
    kapt("com.google.dagger:hilt-compiler:2.48")

    // Coroutines
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")

    // Mapbox
    implementation("com.mapbox.maps:android:11.0.0")

    // ZXing (Barcode)
    implementation("com.google.zxing:core:3.5.2")
    implementation("com.journeyapps:zxing-android-embedded:4.3.0")

    // Coil (Images)
    implementation("io.coil-kt:coil:2.5.0")
```

```
    // WorkManager
    implementation("androidx.work:work-runtime-ktx:2.9.0")
}
```

Anexo B: Dependencias Maven (Backend)

xml

```xml
<!-- pom.xml -->
<dependencies>
    <!-- Spring Boot Starters -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-mail</artifactId>
    </dependency>

    <!-- PostgreSQL -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
    </dependency>

    <!-- JWT -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.3</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.3</version>
    </dependency>

    <!-- Lombok -->
    <dependency>
        <groupId>org.projectlombok</groupId>
```

```
        <artifactId>lombok</artifactId>
    </dependency>

    <!-- Validation -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
</dependencies>
```

RESUMEN EJECUTIVO PARA PRESENTACIÓN

SLIOR es un sistema completo de optimización logística desarrollado como proyecto de fin de ciclo de DAM que demuestra:

✅ **Dominio de tecnologías actuales**: Kotlin, Spring Boot, PostgreSQL, APIs REST
✅ **Arquitectura profesional**: MVVM, Clean Architecture, patrón Repository
✅ **Resolución de problemas reales**: Optimización de rutas, modo offline
✅ **Buenas prácticas**: Git Flow, testing, documentación completa
✅ **Alcance realista**: MVP funcional en 100 horas, con roadmap de evolución

El proyecto está diseñado para ser **presentable ante empresas**, con código limpio, arquitectura escalable y documentación profesional.